Análise de Desempenho das Cláusulas `collapse` e `schedule`

Simulação da Equação de Difusão 2D

Werbert Arles de Souza Barradas

Universidade Federal do Rio Grande do Norte (UFRN)
Disciplina de Programação Paralela - DCA3703

28 de setembro de 2025

Agenda

1. Introdução e Estratégias
2. Implementações
3. Resultados e Análise
4. Conclusão

Contexto e Estratégias Avaliadas

Problema

O custo computacional de simulações científicas está concentrado em **laços aninhados**. O objetivo é analisar o impacto de diferentes estratégias de paralelização de laços em OpenMP para um simulador de difusão 2D, com 6000 repetições(NT=6000).

Versões Analisadas

1. **Serial:** Implementação sequencial para baseline.
2. **Paralelismo Ingênuo:** Múltiplas regiões paralelas (alto overhead).
3. **Otimização Parcial:** Região paralela única, mas **sem** `collapse`.
4. **Otimização Completa:** Região única **com** `collapse(2)` e teste das políticas de `schedule`.

1. Implementação Serial (Baseline)

Propósito

Estabelecer o tempo de execução base (T_{serial}). Todos os laços são sequenciais.

```
// Loop de tempo PRINCIPAL - SERIAL
for (int t = 0; t < NT; t++) {
    // La os espaciais - SERIAIS
    for (int i = 1; i < NX-1; i++) {
        for (int j = 1; j < NY-1; j++) {
            // ... c lculo do Laplaciano ...
        }
    }
}
```

2. Paralelização Ingênua

Estratégia e Gargalo

Aplicar `#pragma omp parallel for` em cada laço de trabalho. **Gargalo:** Alto overhead de criar e destruir threads (fork/join) 3x a cada passo de tempo.

```
// Loop de tempo PRINCIPAL - SERIAL
for (int t = 0; t < NT; t++) {
    // GERA OVERHEAD DE FORK/JOIN AQUI
    #pragma omp parallel for
    for (int i = 1; i < NX-1; i++) { /*...*/ }

    // E AQUI...
    #pragma omp parallel for
    for (int i = 0; i < NX; i++) { /*...*/ }

    // E AQUI NOVAMENTE...
    #pragma omp parallel for
    for (int j = 0; j < NY; j++) { /*...*/ }
}
```

3. Otimização Parcial (sem collapse)

Estratégia e Gargalo

Criar uma **única região paralela** para eliminar o overhead de fork/join. **Gargalo Remanescente:** Sem `collapse`, o OpenMP só distribui as linhas (laço `i`), limitando o balanceamento de carga.

```
// Loop de tempo PRINCIPAL - SERIAL
for (int t = 0; t < NT; t++) {
    // UMA  NICA  CRIA  O DE THREADS AQUI
    #pragma omp parallel
    {
        // AINDA SEM COLLAPSE: paraleliza apenas o la o 'i'
        #pragma omp for
        for (int i = 1; i < NX-1; i++) {
            for (int j = 1; j < NY-1; j++) { /*...*/ }
        }

        #pragma omp for
        for (int i = 0; i < NX; i++) { /*...*/ }
    } // FIM DA REGI O PARALELA
}
```

4. Paralelização Otimizada (com collapse)

Estratégia Final

Manter a região paralela única e adicionar `collapse(2)` para tratar a grade 2D como um único bloco de trabalho, maximizando o potencial de balanceamento de carga.

```
// Loop de tempo PRINCIPAL - SERIAL
for (int t = 0; t < NT; t++) {
    #pragma omp parallel
    {
        // MELHOR BALANCEAMENTO DE CARGA
        #pragma omp for collapse(2) schedule(static)
        for (int i = 1; i < NX-1; i++) {
            for (int j = 1; j < NY-1; j++) { /*...*/ }
        }

        #pragma omp for
        for (int i = 0; i < NX; i++) { /*...*/ }
    }
}
```

Resultados Quantitativos (Tempos de Execução)

Estratégia	N. Threads	Tempo Médio (s)
Serial	**1**	**3.7683**
Paralelo Ingênua	16	3.0730
Otimizado (schedule + static)	16	3.1411
Otimizado (schedule + guided)	16	3.5169
Otimizado (collapse + static)	16	4.0044
Otimizado (collapse + guided)	16	4.3125
Otimizado (collapse + dynamic)	16	43.2871

Tabela: O melhor resultado (16 threads) é apresentado para as versões paralelas.

Comparativo de Desempenho (16 Threads)

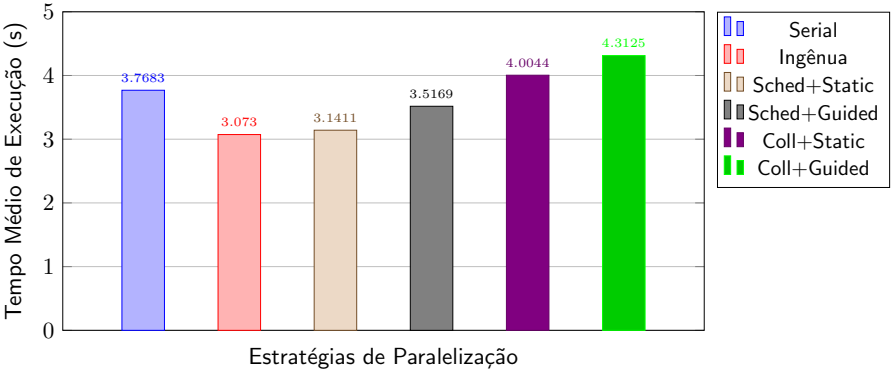

Figura: Comparativo do tempo de execução (16 threads). A estratégia `Sched+Static` apresentou o melhor desempenho.

Avaliação dos Resultados

Por que `static` foi rápido e `dynamic` foi lento?

A política de agendamento deve ser compatível com a natureza da carga de trabalho.

Cenário Ideal: Carga Uniforme

- **Problema:** Cada iteração do laço leva o mesmo tempo.
- **Solução:** `schedule(static)`.
- **Mecanismo:** O trabalho é dividido em blocos iguais **uma única vez**.
- **Resultado: Overhead mínimo**, melhor desempenho.

Cenário Inadequado: Carga Uniforme

- **Problema:** O mesmo, com carga uniforme.
- **Solução Incorreta:** `schedule(dynamic)`.
- **Mecanismo:** Cada thread pega **uma iteração de cada vez**.
- **Resultado: Overhead massivo**, desaceleração.

Conclusões

- **Estratégia > Micro-otimização:** A decisão de **quais laços** paralelizar foi o que gerou o maior ganho de desempenho.
- **O `schedule` é o fator mais crítico:** A comparação entre `static` (melhor desempenho) e `dynamic` (desaceleração acentuada) provou que a escolha do agendamento deve ser adequada à natureza da carga de trabalho.
- **Gargalos de Hardware:**A pequena diferença entre as versões otimizadas sugerem que o gargalo final não é o balanceamento de carga, mas sim a **largura de banda da memória** do sistema.

serial/navier_stokes_simul_serial_tempo.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define NX 512
#define NY 512
#define NT 6000
#define DT 0.001
#define NU 0.01

int main() {
    // Alocar arrays 2D
    double **u = (double**)malloc(NX * sizeof(double*));
    double **v = (double**)malloc(NX * sizeof(double*));
    double **u_new = (double**)malloc(NX * sizeof(double*));
    double **v_new = (double**)malloc(NX * sizeof(double*));

    for (int i = 0; i < NX; i++) {
        u[i] = (double*)malloc(NY * sizeof(double));
        v[i] = (double*)malloc(NY * sizeof(double));
        u_new[i] = (double*)malloc(NY * sizeof(double));
        v_new[i] = (double*)malloc(NY * sizeof(double));
    }

    // Inicialização
    for (int i = 0; i < NX; i++) {
        for (int j = 0; j < NY; j++) {
            u[i][j] = 1.0;
            v[i][j] = 0.0;
            double dx = i - NX/2, dy = j - NY/2;
            double dist = sqrt(dx*dx + dy*dy);
            if (dist < 20.0) {
                u[i][j] += 2.0 * exp(-dist*dist/100.0);
                v[i][j] += 1.5 * exp(-dist*dist/100.0);
            }
        }
    }

    double start_time = omp_get_wtime();

    // Simulação principal
    for (int step = 0; step < NT; step++) {
        for (int i = 1; i < NX-1; i++) {
            for (int j = 1; j < NY-1; j++) {
                double d2u_dx2 = (u[i+1][j] - 2.0*u[i][j] + u[i-1][j]);
                double d2u_dy2 = (u[i][j+1] - 2.0*u[i][j] + u[i][j-1]);
                double d2v_dx2 = (v[i+1][j] - 2.0*v[i][j] + v[i-1][j]);
                double d2v_dy2 = (v[i][j+1] - 2.0*v[i][j] + v[i][j-1]);

```

```
                u_new[i][j] = u[i][j] + DT * NU * (d2u_dx2 + d2u_dy2);
                v_new[i][j] = v[i][j] + DT * NU * (d2v_dx2 + d2v_dy2);
            }
        }

        // Condições de contorno periódicas
        for (int i = 0; i < NX; i++) {
            u_new[i][0] = u_new[i][NY-2];
            u_new[i][NY-1] = u_new[i][1];
            v_new[i][0] = v_new[i][NY-2];
            v_new[i][NY-1] = v_new[i][1];
        }
        for (int j = 0; j < NY; j++) {
            u_new[0][j] = u_new[NX-2][j];
            u_new[NX-1][j] = u_new[1][j];
            v_new[0][j] = v_new[NX-2][j];
            v_new[NX-1][j] = v_new[1][j];
        }

        // Trocar arrays
        double **temp_u = u;
        double **temp_v = v;
        u = u_new;
        v = v_new;
        u_new = temp_u;
        v_new = temp_v;
    }

    double end_time = omp_get_wtime();

    // Saída apenas do tempo
    printf("%.6f\n", end_time - start_time);

    // Liberar memória
    for (int i = 0; i < NX; i++) {
        free(u[i]);
        free(v[i]);
        free(u_new[i]);
        free(v_new[i]);
    }
    free(u);
    free(v);
    free(u_new);
    free(v_new);

    return 0;
}
```

paralelo/navier_stokes_paralelo_ingenua.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define NX 512
#define NY 512
#define NT 6000
#define DT 0.001
#define NU 0.01

int main() {
    // Alocar arrays 2D
    double **u = (double**)malloc(NX * sizeof(double*));
    double **v = (double**)malloc(NX * sizeof(double*));
    double **u_new = (double**)malloc(NX * sizeof(double*));
    double **v_new = (double**)malloc(NX * sizeof(double*));

    for (int i = 0; i < NX; i++) {
        u[i] = (double*)malloc(NY * sizeof(double));
        v[i] = (double*)malloc(NY * sizeof(double));
        u_new[i] = (double*)malloc(NY * sizeof(double));
        v_new[i] = (double*)malloc(NY * sizeof(double));
    }

    // Inicialização (pode ser paralelizada também, mas o ganho é desprezível)
    #pragma omp parallel for
    for (int i = 0; i < NX; i++) {
        for (int j = 0; j < NY; j++) {
            u[i][j] = 1.0;
            v[i][j] = 0.0;
            double dx = i - NX/2, dy = j - NY/2;
            double dist = sqrt(dx*dx + dy*dy);
            if (dist < 20.0) {
                u[i][j] += 2.0 * exp(-dist*dist/100.0);
                v[i][j] += 1.5 * exp(-dist*dist/100.0);
            }
        }
    }

    double start_time = omp_get_wtime();

    // Simulação principal
    for (int step = 0; step < NT; step++) {

        #pragma omp parallel for
        for (int i = 1; i < NX-1; i++) {
            for (int j = 1; j < NY-1; j++) {
                double d2u_dx2 = (u[i+1][j] - 2.0*u[i][j] + u[i-1][j]);
                double d2u_dy2 = (u[i][j+1] - 2.0*u[i][j] + u[i][j-1]);
```

```
                    double d2v_dx2 = (v[i+1][j] - 2.0*v[i][j] + v[i-1][j]);
                    double d2v_dy2 = (v[i][j+1] - 2.0*v[i][j] + v[i][j-1]);

                    u_new[i][j] = u[i][j] + DT * NU * (d2u_dx2 + d2u_dy2);
                    v_new[i][j] = v[i][j] + DT * NU * (d2v_dx2 + d2v_dy2);
                }
            }

            #pragma omp sections
            {
                #pragma omp section
                { // Tarefa A: Contorno Horizontal
                    for (int i = 0; i < NX; i++) {
                        u_new[i][0] = u_new[i][NY-2];
                        u_new[i][NY-1] = u_new[i][1];
                        v_new[i][0] = v_new[i][NY-2];
                        v_new[i][NY-1] = v_new[i][1];
                    }
                }
                #pragma omp section
                { // Tarefa B: Contorno Vertical
                    for (int j = 0; j < NY; j++) {
                        u_new[0][j] = u_new[NX-2][j];
                        u_new[NX-1][j] = u_new[1][j];
                        v_new[0][j] = v_new[NX-2][j];
                        v_new[NX-1][j] = v_new[1][j];
                    }
                }
            } // Barreira implícita no final de 'sections'

            #pragma omp single
            {
                double **temp_u = u;
                double **temp_v = v;
                u = u_new;
                v = v_new;
                u_new = temp_u;
                v_new = temp_v;
            } // Barreira implícita no final de 'single' para garantir que a troca foi feita antes do próximo 'step'

    } // Fim do loop de tempo


    double end_time = omp_get_wtime();

    // Saída apenas do tempo
    printf("%.6f\n", end_time - start_time);

    // Liberar memória
    for (int i = 0; i < NX; i++) {
        free(u[i]);
        free(v[i]);
```

```
        free(u_new[i]);
        free(v_new[i]);
    }
    free(u);
    free(v);
    free(u_new);
    free(v_new);

    return 0;
}
```

paralelo/navier_stokes_paralelo_otm1.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define NX 512
#define NY 512
#define NT 6000
#define DT 0.001
#define NU 0.01

int main() {
    // Alocar arrays 2D
    double **u = (double**)malloc(NX * sizeof(double*));
    double **v = (double**)malloc(NX * sizeof(double*));
    double **u_new = (double**)malloc(NX * sizeof(double*));
    double **v_new = (double**)malloc(NX * sizeof(double*));

    for (int i = 0; i < NX; i++) {
        u[i] = (double*)malloc(NY * sizeof(double));
        v[i] = (double*)malloc(NY * sizeof(double));
        u_new[i] = (double*)malloc(NY * sizeof(double));
        v_new[i] = (double*)malloc(NY * sizeof(double));
    }

    // Inicialização (pode ser paralelizada também, mas o ganho é desprezível)
    #pragma omp parallel for
    for (int i = 0; i < NX; i++) {
        for (int j = 0; j < NY; j++) {
            u[i][j] = 1.0;
            v[i][j] = 0.0;
            double dx = i - NX/2, dy = j - NY/2;
            double dist = sqrt(dx*dx + dy*dy);
            if (dist < 20.0) {
                u[i][j] += 2.0 * exp(-dist*dist/100.0);
                v[i][j] += 1.5 * exp(-dist*dist/100.0);
            }
        }
    }

    double start_time = omp_get_wtime();

    // Simulação principal
    // Criamos a região paralela aqui fora para evitar o overhead de criar/destruir threads 2000 vezes
    for (int step = 0; step < NT; step++) {

        // --- INÍCIO DA REGIÃO PARALELA DENTRO DO PASSO DE TEMPO ---
        // Usamos #pragma omp for e #pragma omp sections para dividir o trabalho entre as threads existentes
        #pragma omp parallel
```

```c
        {
            // 1. CÁLCULO PRINCIPAL: Otimizado com for, collapse e schedule(static)
            // As variáveis de cálculo são privadas para cada thread por padrão dentro do loop
            #pragma omp for schedule(guided)//substituido por static ou dynamic
            for (int i = 1; i < NX-1; i++) {
                for (int j = 1; j < NY-1; j++) {
                    double d2u_dx2 = (u[i+1][j] - 2.0*u[i][j] + u[i-1][j]);
                    double d2u_dy2 = (u[i][j+1] - 2.0*u[i][j] + u[i][j-1]);
                    double d2v_dx2 = (v[i+1][j] - 2.0*v[i][j] + v[i-1][j]);
                    double d2v_dy2 = (v[i][j+1] - 2.0*v[i][j] + v[i][j-1]);

                    u_new[i][j] = u[i][j] + DT * NU * (d2u_dx2 + d2u_dy2);
                    v_new[i][j] = v[i][j] + DT * NU * (d2v_dx2 + d2v_dy2);
                }
            }

            // 2. CONDIÇÕES DE CONTORNO: Usando sections como solicitado
            // A diretiva 'for' acima tem uma barreira implícita, então as threads esperam aqui
            // antes de prosseguir para as seções.
            #pragma omp sections
            {
                #pragma omp section
                { // Tarefa A: Contorno Horizontal
                    for (int i = 0; i < NX; i++) {
                        u_new[i][0] = u_new[i][NY-2];
                        u_new[i][NY-1] = u_new[i][1];
                        v_new[i][0] = v_new[i][NY-2];
                        v_new[i][NY-1] = v_new[i][1];
                    }
                }
                #pragma omp section
                { // Tarefa B: Contorno Vertical
                    for (int j = 0; j < NY; j++) {
                        u_new[0][j] = u_new[NX-2][j];
                        u_new[NX-1][j] = u_new[1][j];
                        v_new[0][j] = v_new[NX-2][j];
                        v_new[NX-1][j] = v_new[1][j];
                    }
                }
            } // Barreira implícita no final de 'sections'

            // 3. TROCAR ARRAYS: Apenas uma thread deve fazer isso
            // O #pragma omp single garante que o bloco de código seja executado por apenas UMA thread.
            #pragma omp single
            {
                double **temp_u = u;
                double **temp_v = v;
                u = u_new;
                v = v_new;
```

```
                u_new = temp_u;
                v_new = temp_v;
            } // Barreira implícita no final de 'single' para garantir que a troca foi feita antes do próximo 'step'

        } // --- FIM DA REGIÃO PARALELA DENTRO DO PASSO DE TEMPO ---
    }

    double end_time = omp_get_wtime();

    // Saída apenas do tempo
    printf("%.6f\n", end_time - start_time);

    // Liberar memória
    for (int i = 0; i < NX; i++) {
        free(u[i]);
        free(v[i]);
        free(u_new[i]);
        free(v_new[i]);
    }
    free(u);
    free(v);
    free(u_new);
    free(v_new);

    return 0;
}
```

paralelo/navier_stokes_paralelo_otm2.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define NX 512
#define NY 512
#define NT 6000
#define DT 0.001
#define NU 0.01

int main() {
    // Alocar arrays 2D
    double **u = (double**)malloc(NX * sizeof(double*));
    double **v = (double**)malloc(NX * sizeof(double*));
    double **u_new = (double**)malloc(NX * sizeof(double*));
    double **v_new = (double**)malloc(NX * sizeof(double*));

    for (int i = 0; i < NX; i++) {
        u[i] = (double*)malloc(NY * sizeof(double));
        v[i] = (double*)malloc(NY * sizeof(double));
        u_new[i] = (double*)malloc(NY * sizeof(double));
        v_new[i] = (double*)malloc(NY * sizeof(double));
    }

    // Inicialização (pode ser paralelizada também, mas o ganho é desprezível)
    #pragma omp parallel for collapse(2) schedule(static)
    for (int i = 0; i < NX; i++) {
        for (int j = 0; j < NY; j++) {
            u[i][j] = 1.0;
            v[i][j] = 0.0;
            double dx = i - NX/2, dy = j - NY/2;
            double dist = sqrt(dx*dx + dy*dy);
            if (dist < 20.0) {
                u[i][j] += 2.0 * exp(-dist*dist/100.0);
                v[i][j] += 1.5 * exp(-dist*dist/100.0);
            }
        }
    }

    double start_time = omp_get_wtime();

    // Simulação principal
    // Criamos a região paralela aqui fora para evitar o overhead de criar/destruir threads 2000 vezes
    for (int step = 0; step < NT; step++) {

        // --- INÍCIO DA REGIÃO PARALELA DENTRO DO PASSO DE TEMPO ---
        // Usamos #pragma omp for e #pragma omp sections para dividir o trabalho entre as threads existentes
        #pragma omp parallel
```

```c
        {
            // 1. CÁLCULO PRINCIPAL: Otimizado com for, collapse e schedule(static)
            // As variáveis de cálculo são privadas para cada thread por padrão dentro do loop
            #pragma omp for collapse(2) schedule(static)//substituido por guided ou dynamic
            for (int i = 1; i < NX-1; i++) {
                for (int j = 1; j < NY-1; j++) {
                    double d2u_dx2 = (u[i+1][j] - 2.0*u[i][j] + u[i-1][j]);
                    double d2u_dy2 = (u[i][j+1] - 2.0*u[i][j] + u[i][j-1]);
                    double d2v_dx2 = (v[i+1][j] - 2.0*v[i][j] + v[i-1][j]);
                    double d2v_dy2 = (v[i][j+1] - 2.0*v[i][j] + v[i][j-1]);

                    u_new[i][j] = u[i][j] + DT * NU * (d2u_dx2 + d2u_dy2);
                    v_new[i][j] = v[i][j] + DT * NU * (d2v_dx2 + d2v_dy2);
                }
            }

            // 2. CONDIÇÕES DE CONTORNO: Usando sections como solicitado
            // A diretiva 'for' acima tem uma barreira implícita, então as threads esperam aqui
            // antes de prosseguir para as seções.
            #pragma omp sections
            {
                #pragma omp section
                { // Tarefa A: Contorno Horizontal
                    for (int i = 0; i < NX; i++) {
                        u_new[i][0] = u_new[i][NY-2];
                        u_new[i][NY-1] = u_new[i][1];
                        v_new[i][0] = v_new[i][NY-2];
                        v_new[i][NY-1] = v_new[i][1];
                    }
                }
                #pragma omp section
                { // Tarefa B: Contorno Vertical
                    for (int j = 0; j < NY; j++) {
                        u_new[0][j] = u_new[NX-2][j];
                        u_new[NX-1][j] = u_new[1][j];
                        v_new[0][j] = v_new[NX-2][j];
                        v_new[NX-1][j] = v_new[1][j];
                    }
                }
            } // Barreira implícita no final de 'sections'

            // 3. TROCAR ARRAYS: Apenas uma thread deve fazer isso
            // O #pragma omp single garante que o bloco de código seja executado por apenas UMA thread.
            #pragma omp single
            {
                double **temp_u = u;
                double **temp_v = v;
                u = u_new;
```

```
                v = v_new;
                u_new = temp_u;
                v_new = temp_v;
            } // Barreira implícita no final de 'single' para garantir que a troca foi feita antes do próximo 'step'

        } // --- FIM DA REGIÃO PARALELA DENTRO DO PASSO DE TEMPO ---
    }

    double end_time = omp_get_wtime();

    // Saída apenas do tempo
    printf("%.6f\n", end_time - start_time);

    // Liberar memória
    for (int i = 0; i < NX; i++) {
        free(u[i]);
        free(v[i]);
        free(u_new[i]);
        free(v_new[i]);
    }
    free(u);
    free(v);
    free(u_new);
    free(v_new);

    return 0;
}
```